WSQ COURSEWARE

Agentic AI for Product Development

Learner Guide

Course Code	**TGS-2024045799**
Technical Skills and Competencies	ICT-TEM-4035-1.1 Automation Management in Product Development
Version	**v7.0**
Release Date	11 September 2026
Training Provider	Tertiary Infotech Academy Pte. Ltd. (UEN 201200696W)
Trainer	Dr Alfred Ang

Controlled document. Verify the version before use.

Document Control

Document	Version	Date	Owner	Change summary
Learner Guide	**6.0**	Prior release	Tertiary Infotech Academy Pte. Ltd.	Legacy chatbot courseware retained as historical reference
Learner Guide	**7.0**	11 September 2026	Tertiary Infotech Academy Pte. Ltd.	Agentic AI product-development remap using both supplied eBooks; visual deck, twelve independent activities, aligned WA-SAQ and PP

Contents

1. Course Overview

This Learner Guide supports an end-to-end, evidence-led agentic AI product-development workflow. Learners frame a validated opportunity, compare agentic builders, define roles and handoff contracts, build and test a thin prototype, then evaluate product value, quality, reliability, cost, risk and human accountability. The detailed procedures are intentionally kept here and out of the visual slide deck.

SAFETY BOUNDARY

All lab data are synthetic. Work only in the supplied lab or a temporary copy. Never place live keys, personal data or proprietary secrets in prompts, files, logs or screenshots; keep destructive and external actions out of scope.

1.1 Learning Outcomes

- LO1 Evaluate agentic AI builders for product research and development, comparing strengths, limitations, resources and skills.
- LO2 Apply optimisation techniques to improve the efficiency, quality and control of agentic product-development workflows.
- LO3 Evaluate the benefits and trade-offs of deploying agentic AI-powered product solutions and recommend a governed path forward.

1.2 Knowledge and Ability Map

Code	Competency statement	Primary evidence
K1	Range of applications of automation technologies across the product-development lifecycle.	Written response
K2	Methods of evaluating the resources and skills required to carry out tasks using automation technologies.	Written response
K3	Concepts pertaining to performance specifications and analysis.	Written response
K4	Best practices in automation.	Written response
K5	Principles of applying automation technologies in products.	Written response
K6	Types of programming skills used with automation technologies.	Written response
A1	Evaluate automation technologies and compare their strengths and limitations.	product artifact, evaluation and decision evidence
A2	Apply optimisation techniques to improve automated-process efficiency and product quality.	product artifact, evaluation and decision evidence
A3	Evaluate benefits and trade-offs of implementing automation in products and businesses.	product artifact, evaluation and decision evidence

1.3 End-to-End Reference Architecture

1. Frame the product opportunity from observable user friction, a measurable baseline and a decision-linked target.
2. Compare agentic builders on a common task using fixed criteria, evidence, readiness and total-cost assumptions.

3. Create a versioned product context packet, outcome specification and explicit non-goals before orchestration.
4. Choose the simplest workflow or agent pattern that fits task variability, risk and required judgment.
5. Define agent roles, tool boundaries, handoff schemas, stop conditions and meaningful human approval gates.
6. Run a frozen evaluation set, classify failures and change one causal layer per optimisation cycle.
7. Combine business value, experience, quality, cost and risk evidence into a PILOT, EXPAND, HOLD or STOP recommendation.

2. Evaluating Agentic AI Tools for Product Research and Development

Identify a product opportunity, map it to a suitable agentic pattern, and compare builders using evidence, readiness and total-cost criteria.

TOPIC OPERATING PRINCIPLE

Every agent handoff uses named fields; every consequential transition has a deterministic gate; every release decision resolves to evidence.

2.1 Agentic AI versus assistants

Mechanism

1. Receive goal
2. Plan work
3. Use tools
4. Adapt from results

goal	plan	tool_call	observation
Required state field for receive goal	Required state field for plan work	Required state field for use tools	Required state field for adapt from results

Evidence and control

MEASURE

Goal completion rate with valid evidence

Failure mode: A chatbot response is mistaken for autonomous task completion.

Control: Require a multi-step loop, tool boundary and stop condition before calling a system agentic.

Acceptance evidence: Trace showing plan, tool result and final decision

2.2 Product lifecycle application map

Mechanism

1. Discover need
2. Define concept
3. Deliver prototype
4. Learn in market

insight	hypothesis	artifact	feedback
Required state field for discover need	Required state field for define concept	Required state field for deliver prototype	Required state field for learn in market

Evidence and control

MEASURE

% lifecycle stages with a named human owner

Failure mode: Automation is applied everywhere without a clear decision or owner.

Control: Map each agent task to one lifecycle decision and accountable role.

Acceptance evidence: Lifecycle map with task-owner pairs

2.3 Opportunity framing

Mechanism

1. Observe friction
2. Name user
3. Quantify impact
4. Frame outcome

problem	user	baseline	target
Required state field for observe friction	Required state field for name user	Required state field for quantify impact	Required state field for frame outcome

Evidence and control

MEASURE

Validated problem statements / total ideas

Failure mode: The team automates an interesting task that is not a material user problem.

Control: Start with observable friction and a measurable outcome, not a tool demonstration.

Acceptance evidence: Opportunity brief with baseline and target

2.4 Jobs-to-be-done lens

Mechanism

1. Capture situation
2. Identify progress
3. Expose obstacle
4. Define success

situation	job	barrier	outcome
Required state field for capture situation	Required state field for identify progress	Required state field for expose obstacle	Required state field for define success

Evidence and control

MEASURE

% research observations linked to a user job

Failure mode: Personas become demographics without decision-relevant needs.

Control: Write needs as progress users seek in a real context.

Acceptance evidence: JTBD statement linked to verbatim research notes

2.5 Research-agent pattern

Mechanism

1. Set question
2. Gather sources
3. Triangulate claims
4. Synthesize insight

question	source	claim	confidence
Required state field for set question	Required state field for gather sources	Required state field for triangulate claims	Required state field for synthesize insight

Evidence and control

MEASURE

% material claims supported by two independent sources

Failure mode: The agent produces polished synthesis from weak or circular evidence.

Control: Require source provenance, claim-level support and uncertainty labels.

Acceptance evidence: Research ledger and cited insight brief

2.6 Competitor intelligence workflow

Mechanism

1. Choose peers
2. Freeze criteria
3. Collect evidence
4. Compare gaps

peer	criterion	evidence	gap
Required state field for choose peers	Required state field for freeze criteria	Required state field for collect evidence	Required state field for compare gaps

Evidence and control

MEASURE

Comparable criteria populated per competitor

Failure mode: Different evidence standards make the comparison misleading.

Control: Freeze the same criteria and observation date before collection.

Acceptance evidence: Timestamped comparison matrix

2.7 Voice-of-customer synthesis

Mechanism

1. Ingest feedback
2. Cluster themes
3. Test outliers
4. Prioritise need

feedback_id	theme	outlier	priority
Required state field for ingest feedback	Required state field for cluster themes	Required state field for test outliers	Required state field for prioritise need

Evidence and control

MEASURE

% themes traceable to source rows

Failure mode: A dominant-sounding summary hides minority but critical needs.

Control: Retain row-level traceability and inspect negative cases separately.

Acceptance evidence: Theme table with supporting and contradicting excerpts

2.8 Persona and assumption map

Mechanism

1. Group evidence
2. Draft persona
3. Label assumptions
4. Plan validation

segment	need	assumption	test
Required state field for group evidence	Required state field for draft persona	Required state field for label assumptions	Required state field for plan validation

Evidence and control

MEASURE

High-risk assumptions with a validation test

Failure mode: Generated personas are treated as facts about real users.

Control: Mark inferred attributes and separate evidence from hypothesis.

Acceptance evidence: Persona card plus assumption register

2.9 Agentic builder landscape

Mechanism

1. Define use case
2. Shortlist builders
3. Run common task
4. Score fit

use_case	builder	test_case	score
Required state field for define use case	Required state field for shortlist builders	Required state field for run common task	Required state field for score fit

Evidence and control

MEASURE

Weighted score on a common golden task

Failure mode: Feature lists replace direct comparison on the intended task.

Control: Run the same input, output and constraints across every candidate.

Acceptance evidence: Paired trial results and recommendation

2.10 Workflow versus autonomous agent

Mechanism

1. Assess variability
2. Assess risk
3. Choose pattern
4. Set override

variability	risk	pattern	override
Required state field for assess variability	Required state field for assess risk	Required state field for choose pattern	Required state field for set override

Evidence and control

MEASURE

% high-risk steps on deterministic paths

Failure mode: Autonomy is added where a predictable workflow would be safer and cheaper.

Control: Use the simplest architecture that satisfies variability and judgment needs.

Acceptance evidence: Pattern decision with rejected alternatives

2.11 Build-buy-partner decision

Mechanism

1. Clarify advantage
2. Estimate capability
3. Model ownership
4. Select posture

advantage	capability	ownership_cost	posture
Required state field for clarify advantage	Required state field for estimate capability	Required state field for model ownership	Required state field for select posture

Evidence and control

MEASURE

Risk-adjusted value over total cost of ownership

Failure mode: Subscription price is compared with in-house build cost while integration and governance are ignored.

Control: Include change, integration, assurance and exit costs.

Acceptance evidence: Three-option decision table with sensitivity range

2.12 Resource and skills readiness

Mechanism

1. List capabilities
2. Assess gaps
3. Plan controls
4. Sequence pilot

data	people	platform	governance
Required state field for list capabilities	Required state field for assess gaps	Required state field for plan controls	Required state field for sequence pilot

Evidence and control

MEASURE

Critical readiness gaps closed before pilot

Failure mode: A tool is selected before data access, ownership and evaluation skills exist.

Control: Gate pilot entry on minimum data, product, technical and risk capabilities.

Acceptance evidence: Readiness heatmap and action owners

2.13 Evaluation decision matrix

Mechanism

1. Weight criteria
2. Score evidence
3. Test sensitivity
4. Recommend

weight	score	sensitivity	decision
Required state field for weight criteria	Required state field for score evidence	Required state field for test sensitivity	Required state field for recommend

Evidence and control

MEASURE
Recommendation stability across plausible weights

Failure mode: A predetermined winner is justified with arbitrary weights.

Control: Agree criteria before testing and show how weight changes affect the result.

Acceptance evidence: Weighted matrix, caveats and decision owner

3. Building and Optimising Agentic Product Development Workflows

Translate product intent into bounded agent roles, specifications, hand-offs, evaluation sets and an iterative prototype workflow that improves with evidence.

TOPIC OPERATING PRINCIPLE

Every agent handoff uses named fields; every consequential transition has a deterministic gate; every release decision resolves to evidence.

3.1 Goal-plan-act-observe loop

Mechanism

1. Interpret goal
2. Choose next step
3. Act through tool
4. Observe state

goal	next_action	tool_result	status
Required state field for interpret goal	Required state field for choose next step	Required state field for act through tool	Required state field for observe state

Evidence and control

MEASURE

% loops ending in a verified stop state

Failure mode: The agent loops without progress or stops on a plausible narrative.

Control: Use explicit budgets, stop conditions and observable state transitions.

Acceptance evidence: Loop trace with terminal status

3.2 Product context packet

Mechanism

1. State vision
2. Name users
3. Add constraints
4. Define evidence

vision	users	constraints	acceptance
Required state field for state vision	Required state field for name users	Required state field for add constraints	Required state field for define evidence

Evidence and control

MEASURE

Required context fields present before execution

Failure mode: The agent fills missing business context with generic assumptions.

Control: Version a compact context packet and expose unknowns explicitly.

Acceptance evidence: Context file and assumption log

3.3 Outcome-focused specification

Mechanism

1. Describe problem
2. Define behavior
3. Set boundaries
4. Specify done

problem	behavior	non_goals	acceptance
Required state field for describe problem	Required state field for define behavior	Required state field for set boundaries	Required state field for specify done

Evidence and control

MEASURE

% acceptance criteria observable by reviewer

Failure mode: The specification dictates implementation while leaving success ambiguous.

Control: Describe what must be true and what must not change.

Acceptance evidence: PRD section with testable criteria

3.4 User stories and acceptance criteria

Mechanism

1. Name actor
2. State intent
3. Explain value
4. Write examples

actor	intent	value	examples
Required state field for name actor	Required state field for state intent	Required state field for explain value	Required state field for write examples

Evidence and control

MEASURE

% stories with positive, edge and failure examples

Failure mode: A story is marked complete when only the happy path works.

Control: Attach concrete examples and negative conditions to each story.

Acceptance evidence: Story map with example-based acceptance

3.5 Journey-map orchestration

Mechanism

1. Map stages
2. Locate friction
3. Assign agent role
4. Keep human gate

stage	friction	agent_role	gate
Required state field for map stages	Required state field for locate friction	Required state field for assign agent role	Required state field for keep human gate

Evidence and control

MEASURE

% high-consequence moments with human control

Failure mode: The workflow optimises speed while weakening trust at critical moments.

Control: Preserve recovery, explanation and control at consequential journey points.

Acceptance evidence: Journey map with agent and human swimlanes

3.6 Single-agent baseline

Mechanism

1. Bound one goal
2. Connect minimal tools
3. Run baseline
4. Measure gaps

goal	tools	baseline	gaps
Required state field for bound one goal	Required state field for connect minimal tools	Required state field for run baseline	Required state field for measure gaps

Evidence and control

MEASURE

Accepted tasks per 20-case evaluation set

Failure mode: A multi-agent design is introduced before a simpler baseline is measured.

Control: Prove the minimum viable agent before adding roles.

Acceptance evidence: Baseline run log and failure summary

3.7 Multi-agent role design

Mechanism

1. Decompose domains
2. Assign role
3. Limit authority
4. Define coordinator

domain	role	authority	coordinator
Required state field for decompose domains	Required state field for assign role	Required state field for limit authority	Required state field for define coordinator

Evidence and control

MEASURE

Handoffs that add unique value / total handoffs

Failure mode: Multiple agents duplicate work and amplify inconsistent assumptions.

Control: Give each role distinct inputs, output schema and decision rights.

Acceptance evidence: Role charter and responsibility matrix

3.8 Handoff contract

Mechanism

1. Validate input
2. Perform task
3. Emit schema
4. Acknowledge receipt

input_schema	task	output_schema	ack
Required state field for validate input	Required state field for perform task	Required state field for emit schema	Required state field for acknowledge receipt

Evidence and control

MEASURE

% handoffs passing schema validation

Failure mode: Free-form outputs break downstream steps or lose provenance.

Control: Use stable identifiers, structured outputs and fail-closed validation.

Acceptance evidence: Handoff record with validation status

3.9 Tool and retrieval grounding

Mechanism

1. Identify need
2. Select source
3. Retrieve context
4. Cite evidence

need	source	context	citation
Required state field for identify need	Required state field for select source	Required state field for retrieve context	Required state field for cite evidence

Evidence and control

MEASURE

% answer claims grounded in approved sources

Failure mode: The agent uses stale, irrelevant or untrusted material as product truth.

Control: Constrain sources, retain citations and test retrieval misses.

Acceptance evidence: Retrieved evidence linked to output claims

3.10 Prompt contract design

Mechanism

1. State objective
2. Provide context
3. Set constraints
4. Define format

objective	context	constraints	output_schema
Required state field for state objective	Required state field for provide context	Required state field for set constraints	Required state field for define format

Evidence and control

MEASURE

First-pass outputs meeting all contract fields

Failure mode: A long prompt hides conflicting instructions and undefined priorities.

Control: Separate goal, context, rules, examples and output contract.

Acceptance evidence: Prompt version and conformance checklist

3.11 Evaluation-set engineering

Mechanism

1. Sample real tasks
2. Add edge cases
3. Freeze expected traits
4. Score runs

task_id	input	expected	score
Required state field for sample real tasks	Required state field for add edge cases	Required state field for freeze expected traits	Required state field for score runs

Evidence and control

MEASURE

Coverage across core, edge and adversarial cases

Failure mode: Teams judge prototypes from one impressive demonstration.

Control: Build a representative frozen set before optimisation.

Acceptance evidence: Versioned evaluation CSV and score report

3.12 Iteration and error taxonomy

Mechanism

1. Classify failure
2. Find causal layer
3. Apply one change
4. Re-run set

failure_type	layer	change	delta
Required state field for classify failure	Required state field for find causal layer	Required state field for apply one change	Required state field for re-run set

Evidence and control

MEASURE

Improvement on failed cases without regression

Failure mode: Several prompt, model and data changes are bundled, obscuring causality.

Control: Change one causal layer per experiment and re-run the full frozen set.

Acceptance evidence: Experiment ledger with before-after deltas

3.13 Prototype-to-learning loop

Mechanism

1. Build thin slice
2. Expose to users
3. Capture signal
4. Revise hypothesis

prototype	test	signal	hypothesis
Required state field for build thin slice	Required state field for expose to users	Required state field for capture signal	Required state field for revise hypothesis

Evidence and control

MEASURE

Validated assumptions per prototype cycle

Failure mode: Prototype polish becomes the goal instead of learning.

Control: Time-box builds and define the decision each prototype must unlock.

Acceptance evidence: Experiment brief and go-pivot-stop decision

4. Deploying and Evaluating Agentic AI-Powered Product Solutions

Design a deployment posture that preserves human control, security, observability, cost discipline and a production feedback loop.

TOPIC OPERATING PRINCIPLE

Every agent handoff uses named fields; every consequential transition has a deterministic gate; every release decision resolves to evidence.

4.1 Prototype-to-production gate

Mechanism

1. Validate value
2. Validate behavior
3. Assess operations
4. Approve handoff

value	evals	operability	decision
Required state field for validate value	Required state field for validate behavior	Required state field for assess operations	Required state field for approve handoff

Evidence and control

MEASURE

Required gates passed before production investment

Failure mode: A working prototype is treated as production-ready software.

Control: Separate learning code from supported production architecture and evidence.

Acceptance evidence: Handoff pack with eval set, edge cases and prompt strategy

4.2 Deployment channel fit

Mechanism

1. Map user context
2. Assess channel limits
3. Design fallback
4. Select channel

context	channel	fallback	choice
Required state field for map user context	Required state field for assess channel limits	Required state field for design fallback	Required state field for select channel

Evidence and control

MEASURE

Task completion and recovery rate by channel

Failure mode: The easiest integration is chosen despite poor user context or control.

Control: Compare web, workspace, messaging and embedded channels on task and risk.

Acceptance evidence: Channel matrix and fallback path

4.3 Reference deployment architecture

Mechanism

1. Receive request
2. Orchestrate agent
3. Access tools
4. Record outcome

request_id	orchestrator	tool_policy	audit_event
Required state field for receive request	Required state field for orchestrate agent	Required state field for access tools	Required state field for record outcome

Evidence and control

MEASURE

% transactions traceable end to end

Failure mode: Model calls, tool actions and user decisions cannot be correlated.

Control: Carry a stable request identifier through every component.

Acceptance evidence: Architecture diagram and correlated trace

4.4 API and integration contract

Mechanism

1. Authenticate caller
2. Validate request
3. Execute bounded action
4. Return status

identity	schema	action	status
Required state field for authenticate caller	Required state field for validate request	Required state field for execute bounded action	Required state field for return status

Evidence and control

MEASURE

% requests with schema-valid input and explicit outcome

Failure mode: Loose integrations permit malformed or over-broad actions.

Control: Use typed schemas, idempotency and explicit error states.

Acceptance evidence: API contract and negative-test results

4.5 Programming skill mix

Mechanism

1. Specify behavior
2. Read interfaces
3. Test integration
4. Operate service

prompting	api	testing	operations
Required state field for specify behavior	Required state field for read interfaces	Required state field for test integration	Required state field for operate service

Evidence and control

MEASURE

Critical skill roles assigned with accountable owners

Failure mode: Natural-language prototyping is assumed to replace integration and operational expertise.

Control: Plan the product, prompt, data, integration, testing and operations skills explicitly.

Acceptance evidence: Skills matrix and gap plan

4.6 Security and privacy boundary

Mechanism

1. Classify data
2. Minimise access
3. Protect tools
4. Audit use

data_class	permission	tool_policy	audit
Required state field for classify data	Required state field for minimise access	Required state field for protect tools	Required state field for audit use

Evidence and control

MEASURE

% sensitive paths protected by least privilege

Failure mode: Prompt injection or excess permissions turn untrusted content into consequential actions.

Control: Layer input controls, tool allowlists, approval and monitoring.

Acceptance evidence: Threat model, permission map and tested controls

4.7 Human oversight design

Mechanism

1. Classify consequence
2. Set confidence gate
3. Route exception
4. Record decision

risk_class	threshold	reviewer	decision
Required state field for classify consequence	Required state field for set confidence gate	Required state field for route exception	Required state field for record decision

Evidence and control

MEASURE

% consequential actions with meaningful human review

Failure mode: A nominal approval step becomes routine click-through.

Control: Show the evidence and consequence at the point of decision.

Acceptance evidence: Approval log tied to current output

4.8 Observability and product KPIs

Mechanism

1. Instrument event
2. Compute metric
3. Segment failure
4. Trigger action

event	metric	segment	action
Required state field for instrument event	Required state field for compute metric	Required state field for segment failure	Required state field for trigger action

Evidence and control

MEASURE

North-star outcome plus quality, cost and risk guardrails

Failure mode: Usage growth masks falling answer quality or rising correction work.

Control: Pair business outcomes with reliability, safety and experience measures.

Acceptance evidence: Metric tree and alert ownership

4.9 Quality, latency and cost frontier

Mechanism

1. Measure quality
2. Measure latency
3. Load full cost
4. Choose frontier

quality	latency	cost	posture
Required state field for measure quality	Required state field for measure latency	Required state field for load full cost	Required state field for choose frontier

Evidence and control

MEASURE

Accepted outcome cost at target response time

Failure mode: The cheapest or fastest model is chosen without quality-adjusted comparison.

Control: Compare only configurations meeting minimum quality and risk thresholds.

Acceptance evidence: Pareto chart and selected operating point

4.10 Production evaluation loop

Mechanism

1. Sample traffic
2. Score outcomes
3. Review failures
4. Release repair

sample	score	failure	release
Required state field for sample traffic	Required state field for score outcomes	Required state field for review failures	Required state field for release repair

Evidence and control

MEASURE

% production samples scored and actioned

Failure mode: Offline evals remain green while live inputs drift.

Control: Combine frozen regression sets with representative live sampling.

Acceptance evidence: Eval dashboard and linked repair record

4.11 Rollout and change management

Mechanism

1. Pilot cohort
2. Train users
3. Expand gradually
4. Retire safely

cohort	enablement	gate	rollback
Required state field for pilot cohort	Required state field for train users	Required state field for expand gradually	Required state field for retire safely

Evidence and control

MEASURE

Cohorts expanded only after approved thresholds

Failure mode: A big-bang launch creates unmanaged behavior and support load.

Control: Use staged rollout, telemetry, support playbooks and rollback criteria.

Acceptance evidence: Rollout plan with go/no-go gates

4.12 Feedback and graceful failure

Mechanism

1. Detect uncertainty
2. Explain limit
3. Offer recovery
4. Learn from feedback

uncertainty	message	recovery	feedback
Required state field for detect uncertainty	Required state field for explain limit	Required state field for offer recovery	Required state field for learn from feedback

Evidence and control

MEASURE

Successful recovery rate after agent failure

Failure mode: The product hides uncertainty or traps users in repeated bad answers.

Control: Design honest limits, user control and a path to human help.

Acceptance evidence: Failure-state prototype and feedback taxonomy

4.13 Benefit-trade-off recommendation

Mechanism

1. Quantify benefit
2. Price risk
3. Test scenarios
4. Recommend posture

benefit	risk	scenario	recommendation
Required state field for quantify benefit	Required state field for price risk	Required state field for test scenarios	Required state field for recommend posture

Evidence and control

MEASURE

Net value remains positive under downside scenarios

Failure mode: The business case counts automation savings but excludes oversight, incidents and change cost.

Control: Use risk-adjusted scenarios and issue a pilot, expand, hold or stop decision.

Acceptance evidence: Executive decision memo with conditions and owner

5. Hands-on Activity Guide

Each activity is self-contained and includes a detailed README/PDF, YAML brief, prompt, mock CSV/JSON data, worked solution, verifier and evidence checklist.

5.1 Activity 01: Frame a Product Opportunity

OBJECTIVE

Turn synthetic stakeholder notes into a measurable agentic-AI opportunity brief.

Folder contents

- activities/activity-01-opportunity-brief/README.md — detailed procedure
- README.pdf — detailed activity procedure and troubleshooting
- brief.yaml and prompt.md — machine-readable scenario and bounded task contract
- data/mock-data.csv and data/mock-context.json — synthetic inputs with source IDs
- scripts/verify_activity.py, solution/expected-output.json and evidence/checklist.md

Detailed procedure

1. Open the individual activity folder and read README.pdf, brief.yaml, prompt.md and evidence/checklist.md before starting.
2. Inspect the supplied mock CSV/JSON records and note their source IDs, observation date, limitations and missing fields.
3. Restate the product decision this activity must unlock and the observable acceptance rule.
4. Use prompt.md with an approved agentic AI builder; keep the role, data, tool and authority boundaries explicit.
5. Capture the plan, intermediate claims, handoff state and final artifact without introducing live or confidential data.
6. Trace every material claim to the supplied source rows and label inference, uncertainty and contradiction.
7. Review the artifact at the human gate; reject unsupported claims, unowned actions and uncontrolled automation.
8. Run `python3 scripts/verify_activity.py` and retain its JSON result with the activity evidence.
9. Compare the result against the worked solution and diagnose one controlled failure without copying the solution.
10. Complete the evidence checklist and make a PASS, REPAIR or STOP decision against the acceptance rule.

Lab-specific workflow checks

Sequence	Workflow stage	What to inspect
1	**Inspect notes**	Inputs, changed state, permissions, observable status and retained verification evidence
2	**Name user**	Inputs, changed state, permissions, observable status and retained verification evidence

3	**Quantify friction**	Inputs, changed state, permissions, observable status and retained verification evidence
4	**Define outcome**	Inputs, changed state, permissions, observable status and retained verification evidence
5	**List assumptions**	Inputs, changed state, permissions, observable status and retained verification evidence
6	**Set validation**	Inputs, changed state, permissions, observable status and retained verification evidence

Prompt contract

> ***Analyse the supplied stakeholder notes. Draft a product opportunity statement, baseline, target outcome and ranked assumptions. Distinguish evidence from inference.***

Acceptance check

PASS ONLY WHEN

Every claim traces to a source row; the outcome is measurable; high-risk assumptions have named tests.

Troubleshooting

Observed issue	Likely cause	Remedial action
The agent invents a customer or market claim	Source IDs were omitted from the prompt or output contract	Repair the prompt and require row-level citations plus an uncertainty label.
The workflow recommends a tool without comparison	Criteria or common test case were not frozen	Run every option on the same mock input and apply the stated weights.
Output is plausible but unsupported	Acceptance omitted observable evidence	Require the claim ledger, scored rubric and named decision owner before acceptance.
Verifier reports a missing artifact	The required JSON/YAML field or output file is absent	Use brief.yaml as the contract, add only the missing field and re-run the verifier.

5.2 Activity 02: Compare Agentic AI Builders

OBJECTIVE

Run a common product-research task across three synthetic builder profiles and select a fit-for-purpose option.

Folder contents

- activities/activity-02-builder-comparison/README.md — detailed procedure
- README.pdf — detailed activity procedure and troubleshooting
- brief.yaml and prompt.md — machine-readable scenario and bounded task contract
- data/mock-data.csv and data/mock-context.json — synthetic inputs with source IDs
- scripts/verify_activity.py, solution/expected-output.json and evidence/checklist.md

Detailed procedure

1. Open the individual activity folder and read README.pdf, brief.yaml, prompt.md and evidence/checklist.md before starting.
2. Inspect the supplied mock CSV/JSON records and note their source IDs, observation date, limitations and missing fields.
3. Restate the product decision this activity must unlock and the observable acceptance rule.
4. Use prompt.md with an approved agentic AI builder; keep the role, data, tool and authority boundaries explicit.
5. Capture the plan, intermediate claims, handoff state and final artifact without introducing live or confidential data.
6. Trace every material claim to the supplied source rows and label inference, uncertainty and contradiction.
7. Review the artifact at the human gate; reject unsupported claims, unowned actions and uncontrolled automation.
8. Run `python3 scripts/verify_activity.py` and retain its JSON result with the activity evidence.
9. Compare the result against the worked solution and diagnose one controlled failure without copying the solution.
10. Complete the evidence checklist and make a PASS, REPAIR or STOP decision against the acceptance rule.

Lab-specific workflow checks

Sequence	Workflow stage	What to inspect
1	**Freeze task**	Inputs, changed state, permissions, observable status and retained verification evidence
2	**Set criteria**	Inputs, changed state, permissions, observable status and retained verification evidence
3	**Score builders**	Inputs, changed state, permissions, observable status and retained verification evidence
4	**Apply weights**	Inputs, changed state, permissions, observable status and retained

		verification evidence
5	**Test sensitivity**	Inputs, changed state, permissions, observable status and retained verification evidence
6	**Recommend**	Inputs, changed state, permissions, observable status and retained verification evidence

Prompt contract

> ***Use the supplied profiles and trial results to score three builders. Explain the recommendation and how it changes under an alternate weighting.***

Acceptance check

PASS ONLY WHEN

Scores use one common task, weights total 100%, and the recommendation survives or explains sensitivity.

Troubleshooting

Observed issue	Likely cause	Remedial action
The agent invents a customer or market claim	Source IDs were omitted from the prompt or output contract	Repair the prompt and require row-level citations plus an uncertainty label.
The workflow recommends a tool without comparison	Criteria or common test case were not frozen	Run every option on the same mock input and apply the stated weights.
Output is plausible but unsupported	Acceptance omitted observable evidence	Require the claim ledger, scored rubric and named decision owner before acceptance.
Verifier reports a missing artifact	The required JSON/YAML field or output file is absent	Use brief.yaml as the contract, add only the missing field and re-run the verifier.

5.3 Activity 03: Build a Research Evidence Pack

OBJECTIVE

Transform mock market and competitor records into a traceable, uncertainty-aware insight brief.

Folder contents

- activities/activity-03-research-evidence-pack/README.md — detailed procedure
- README.pdf — detailed activity procedure and troubleshooting
- brief.yaml and prompt.md — machine-readable scenario and bounded task contract
- data/mock-data.csv and data/mock-context.json — synthetic inputs with source IDs
- scripts/verify_activity.py, solution/expected-output.json and evidence/checklist.md

Detailed procedure

1. Open the individual activity folder and read README.pdf, brief.yaml, prompt.md and evidence/checklist.md before starting.
2. Inspect the supplied mock CSV/JSON records and note their source IDs, observation date, limitations and missing fields.
3. Restate the product decision this activity must unlock and the observable acceptance rule.
4. Use prompt.md with an approved agentic AI builder; keep the role, data, tool and authority boundaries explicit.
5. Capture the plan, intermediate claims, handoff state and final artifact without introducing live or confidential data.
6. Trace every material claim to the supplied source rows and label inference, uncertainty and contradiction.
7. Review the artifact at the human gate; reject unsupported claims, unowned actions and uncontrolled automation.
8. Run `python3 scripts/verify_activity.py` and retain its JSON result with the activity evidence.
9. Compare the result against the worked solution and diagnose one controlled failure without copying the solution.
10. Complete the evidence checklist and make a PASS, REPAIR or STOP decision against the acceptance rule.

Lab-specific workflow checks

Sequence	Workflow stage	What to inspect
1	**Load records**	Inputs, changed state, permissions, observable status and retained verification evidence
2	**Normalise facts**	Inputs, changed state, permissions, observable status and retained verification evidence
3	**Triangulate claims**	Inputs, changed state, permissions, observable status and retained verification evidence
4	**Flag conflict**	Inputs, changed state, permissions, observable status and retained verification evidence

5	**Rank insight**	Inputs, changed state, permissions, observable status and retained verification evidence
6	**Cite sources**	Inputs, changed state, permissions, observable status and retained verification evidence

Prompt contract

> ***Create a claim ledger from the supplied mock sources. Keep contradictions visible and assign a confidence level with rationale.***

Acceptance check

PASS ONLY WHEN

Material insights cite at least two records or explicitly state single-source uncertainty.

Troubleshooting

Observed issue	Likely cause	Remedial action
The agent invents a customer or market claim	Source IDs were omitted from the prompt or output contract	Repair the prompt and require row-level citations plus an uncertainty label.
The workflow recommends a tool without comparison	Criteria or common test case were not frozen	Run every option on the same mock input and apply the stated weights.
Output is plausible but unsupported	Acceptance omitted observable evidence	Require the claim ledger, scored rubric and named decision owner before acceptance.
Verifier reports a missing artifact	The required JSON/YAML field or output file is absent	Use brief.yaml as the contract, add only the missing field and re-run the verifier.

5.4 Activity 04: Create Persona and JTBD Map

OBJECTIVE

Derive evidence-based personas and jobs-to-be-done without presenting generated assumptions as facts.

Folder contents

- activities/activity-04-persona-jtbd-map/README.md — detailed procedure
- README.pdf — detailed activity procedure and troubleshooting
- brief.yaml and prompt.md — machine-readable scenario and bounded task contract
- data/mock-data.csv and data/mock-context.json — synthetic inputs with source IDs
- scripts/verify_activity.py, solution/expected-output.json and evidence/checklist.md

Detailed procedure

1. Open the individual activity folder and read README.pdf, brief.yaml, prompt.md and evidence/checklist.md before starting.
2. Inspect the supplied mock CSV/JSON records and note their source IDs, observation date, limitations and missing fields.
3. Restate the product decision this activity must unlock and the observable acceptance rule.
4. Use prompt.md with an approved agentic AI builder; keep the role, data, tool and authority boundaries explicit.
5. Capture the plan, intermediate claims, handoff state and final artifact without introducing live or confidential data.
6. Trace every material claim to the supplied source rows and label inference, uncertainty and contradiction.
7. Review the artifact at the human gate; reject unsupported claims, unowned actions and uncontrolled automation.
8. Run `python3 scripts/verify_activity.py` and retain its JSON result with the activity evidence.
9. Compare the result against the worked solution and diagnose one controlled failure without copying the solution.
10. Complete the evidence checklist and make a PASS, REPAIR or STOP decision against the acceptance rule.

Lab-specific workflow checks

Sequence	Workflow stage	What to inspect
1	**Cluster needs**	Inputs, changed state, permissions, observable status and retained verification evidence
2	**Draft persona**	Inputs, changed state, permissions, observable status and retained verification evidence
3	**Write JTBD**	Inputs, changed state, permissions, observable status and retained verification evidence
4	**Label inference**	Inputs, changed state, permissions, observable status and retained

		verification evidence
5	**Rank risk**	Inputs, changed state, permissions, observable status and retained verification evidence
6	**Plan test**	Inputs, changed state, permissions, observable status and retained verification evidence

Prompt contract

> ***Use the interview snippets to create two personas and JTBD statements. Mark every unsupported attribute as an assumption and propose a test.***

Acceptance check

PASS ONLY WHEN

Persona needs trace to source IDs; assumptions are labelled; each risky assumption has a feasible validation test.

Troubleshooting

Observed issue	Likely cause	Remedial action
The agent invents a customer or market claim	Source IDs were omitted from the prompt or output contract	Repair the prompt and require row-level citations plus an uncertainty label.
The workflow recommends a tool without comparison	Criteria or common test case were not frozen	Run every option on the same mock input and apply the stated weights.
Output is plausible but unsupported	Acceptance omitted observable evidence	Require the claim ledger, scored rubric and named decision owner before acceptance.
Verifier reports a missing artifact	The required JSON/YAML field or output file is absent	Use brief.yaml as the contract, add only the missing field and re-run the verifier.

5.5 Activity 05: Design an Agent Workflow Canvas

OBJECTIVE

Map a product-discovery task into an agentic loop with tools, state, stop conditions and human gates.

Folder contents

- activities/activity-05-agent-workflow-canvas/README.md — detailed procedure
- README.pdf — detailed activity procedure and troubleshooting
- brief.yaml and prompt.md — machine-readable scenario and bounded task contract
- data/mock-data.csv and data/mock-context.json — synthetic inputs with source IDs
- scripts/verify_activity.py, solution/expected-output.json and evidence/checklist.md

Detailed procedure

1. Open the individual activity folder and read README.pdf, brief.yaml, prompt.md and evidence/checklist.md before starting.
2. Inspect the supplied mock CSV/JSON records and note their source IDs, observation date, limitations and missing fields.
3. Restate the product decision this activity must unlock and the observable acceptance rule.
4. Use prompt.md with an approved agentic AI builder; keep the role, data, tool and authority boundaries explicit.
5. Capture the plan, intermediate claims, handoff state and final artifact without introducing live or confidential data.
6. Trace every material claim to the supplied source rows and label inference, uncertainty and contradiction.
7. Review the artifact at the human gate; reject unsupported claims, unowned actions and uncontrolled automation.
8. Run `python3 scripts/verify_activity.py` and retain its JSON result with the activity evidence.
9. Compare the result against the worked solution and diagnose one controlled failure without copying the solution.
10. Complete the evidence checklist and make a PASS, REPAIR or STOP decision against the acceptance rule.

Lab-specific workflow checks

Sequence	Workflow stage	What to inspect
1	**Define goal**	Inputs, changed state, permissions, observable status and retained verification evidence
2	**Map stages**	Inputs, changed state, permissions, observable status and retained verification evidence
3	**Assign tools**	Inputs, changed state, permissions, observable status and retained verification evidence
4	**Set schemas**	Inputs, changed state, permissions, observable status and retained verification evidence

5	**Add gates**	Inputs, changed state, permissions, observable status and retained verification evidence
6	**Set stop**	Inputs, changed state, permissions, observable status and retained verification evidence

Prompt contract

> ***Design the smallest workflow that can complete the supplied discovery task. Specify inputs, outputs, tools, failure paths and human decisions.***

Acceptance check

PASS ONLY WHEN

Every stage has an owner and observable output; consequential actions stop for human review.

Troubleshooting

Observed issue	Likely cause	Remedial action
The agent invents a customer or market claim	Source IDs were omitted from the prompt or output contract	Repair the prompt and require row-level citations plus an uncertainty label.
The workflow recommends a tool without comparison	Criteria or common test case were not frozen	Run every option on the same mock input and apply the stated weights.
Output is plausible but unsupported	Acceptance omitted observable evidence	Require the claim ledger, scored rubric and named decision owner before acceptance.
Verifier reports a missing artifact	The required JSON/YAML field or output file is absent	Use brief.yaml as the contract, add only the missing field and re-run the verifier.

5.6 Activity 06: Create an AI-Ready PRD

OBJECTIVE

Convert an opportunity brief into outcome-focused requirements, user stories and acceptance examples.

Folder contents

- activities/activity-06-prd-and-user-stories/README.md — detailed procedure
- README.pdf — detailed activity procedure and troubleshooting
- brief.yaml and prompt.md — machine-readable scenario and bounded task contract
- data/mock-data.csv and data/mock-context.json — synthetic inputs with source IDs
- scripts/verify_activity.py, solution/expected-output.json and evidence/checklist.md

Detailed procedure

1. Open the individual activity folder and read README.pdf, brief.yaml, prompt.md and evidence/checklist.md before starting.
2. Inspect the supplied mock CSV/JSON records and note their source IDs, observation date, limitations and missing fields.
3. Restate the product decision this activity must unlock and the observable acceptance rule.
4. Use prompt.md with an approved agentic AI builder; keep the role, data, tool and authority boundaries explicit.
5. Capture the plan, intermediate claims, handoff state and final artifact without introducing live or confidential data.
6. Trace every material claim to the supplied source rows and label inference, uncertainty and contradiction.
7. Review the artifact at the human gate; reject unsupported claims, unowned actions and uncontrolled automation.
8. Run `python3 scripts/verify_activity.py` and retain its JSON result with the activity evidence.
9. Compare the result against the worked solution and diagnose one controlled failure without copying the solution.
10. Complete the evidence checklist and make a PASS, REPAIR or STOP decision against the acceptance rule.

Lab-specific workflow checks

Sequence	Workflow stage	What to inspect
1	**Restate problem**	Inputs, changed state, permissions, observable status and retained verification evidence
2	**Define behavior**	Inputs, changed state, permissions, observable status and retained verification evidence
3	**Write stories**	Inputs, changed state, permissions, observable status and retained verification evidence
4	**Add examples**	Inputs, changed state, permissions, observable status and retained

		verification evidence
5	**Set non-goals**	Inputs, changed state, permissions, observable status and retained verification evidence
6	**Review gaps**	Inputs, changed state, permissions, observable status and retained verification evidence

Prompt contract

> ***Draft a lean PRD from the supplied opportunity brief. Focus on what must be true, include non-goals, and add positive, edge and failure examples.***

Acceptance check

PASS ONLY WHEN

Requirements are testable, non-goals are explicit, and every priority story has three acceptance examples.

Troubleshooting

Observed issue	Likely cause	Remedial action
The agent invents a customer or market claim	Source IDs were omitted from the prompt or output contract	Repair the prompt and require row-level citations plus an uncertainty label.
The workflow recommends a tool without comparison	Criteria or common test case were not frozen	Run every option on the same mock input and apply the stated weights.
Output is plausible but unsupported	Acceptance omitted observable evidence	Require the claim ledger, scored rubric and named decision owner before acceptance.
Verifier reports a missing artifact	The required JSON/YAML field or output file is absent	Use brief.yaml as the contract, add only the missing field and re-run the verifier.

5.7 Activity 07: Engineer a Prompt and Evaluation Set

OBJECTIVE

Create a structured prompt contract and a frozen evaluation set for a product-feedback agent.

Folder contents

- activities/activity-07-prompt-evaluation-set/README.md — detailed procedure
- README.pdf — detailed activity procedure and troubleshooting
- brief.yaml and prompt.md — machine-readable scenario and bounded task contract
- data/mock-data.csv and data/mock-context.json — synthetic inputs with source IDs
- scripts/verify_activity.py, solution/expected-output.json and evidence/checklist.md

Detailed procedure

1. Open the individual activity folder and read README.pdf, brief.yaml, prompt.md and evidence/checklist.md before starting.
2. Inspect the supplied mock CSV/JSON records and note their source IDs, observation date, limitations and missing fields.
3. Restate the product decision this activity must unlock and the observable acceptance rule.
4. Use prompt.md with an approved agentic AI builder; keep the role, data, tool and authority boundaries explicit.
5. Capture the plan, intermediate claims, handoff state and final artifact without introducing live or confidential data.
6. Trace every material claim to the supplied source rows and label inference, uncertainty and contradiction.
7. Review the artifact at the human gate; reject unsupported claims, unowned actions and uncontrolled automation.
8. Run `python3 scripts/verify_activity.py` and retain its JSON result with the activity evidence.
9. Compare the result against the worked solution and diagnose one controlled failure without copying the solution.
10. Complete the evidence checklist and make a PASS, REPAIR or STOP decision against the acceptance rule.

Lab-specific workflow checks

Sequence	Workflow stage	What to inspect
1	**Define contract**	Inputs, changed state, permissions, observable status and retained verification evidence
2	**Sample cases**	Inputs, changed state, permissions, observable status and retained verification evidence
3	**Add edges**	Inputs, changed state, permissions, observable status and retained verification evidence
4	**Freeze rubric**	Inputs, changed state, permissions, observable status and retained verification evidence

5	**Run baseline**	Inputs, changed state, permissions, observable status and retained verification evidence
6	**Record gaps**	Inputs, changed state, permissions, observable status and retained verification evidence

Prompt contract

> ***Build a prompt contract and classify the supplied cases into core, edge and adversarial segments. Define observable scoring rules before optimisation.***

Acceptance check

PASS ONLY WHEN

The evaluation set covers all segments; scoring rules are reproducible; baseline failures are retained.

Troubleshooting

Observed issue	Likely cause	Remedial action
The agent invents a customer or market claim	Source IDs were omitted from the prompt or output contract	Repair the prompt and require row-level citations plus an uncertainty label.
The workflow recommends a tool without comparison	Criteria or common test case were not frozen	Run every option on the same mock input and apply the stated weights.
Output is plausible but unsupported	Acceptance omitted observable evidence	Require the claim ledger, scored rubric and named decision owner before acceptance.
Verifier reports a missing artifact	The required JSON/YAML field or output file is absent	Use brief.yaml as the contract, add only the missing field and re-run the verifier.

5.8 Activity 08: Run a Prototype Experiment

OBJECTIVE

Evaluate a synthetic agent prototype, classify failures and recommend one causal improvement.

Folder contents

- activities/activity-08-prototype-experiment/README.md — detailed procedure
- README.pdf — detailed activity procedure and troubleshooting
- brief.yaml and prompt.md — machine-readable scenario and bounded task contract
- data/mock-data.csv and data/mock-context.json — synthetic inputs with source IDs
- scripts/verify_activity.py, solution/expected-output.json and evidence/checklist.md

Detailed procedure

1. Open the individual activity folder and read README.pdf, brief.yaml, prompt.md and evidence/checklist.md before starting.
2. Inspect the supplied mock CSV/JSON records and note their source IDs, observation date, limitations and missing fields.
3. Restate the product decision this activity must unlock and the observable acceptance rule.
4. Use prompt.md with an approved agentic AI builder; keep the role, data, tool and authority boundaries explicit.
5. Capture the plan, intermediate claims, handoff state and final artifact without introducing live or confidential data.
6. Trace every material claim to the supplied source rows and label inference, uncertainty and contradiction.
7. Review the artifact at the human gate; reject unsupported claims, unowned actions and uncontrolled automation.
8. Run `python3 scripts/verify_activity.py` and retain its JSON result with the activity evidence.
9. Compare the result against the worked solution and diagnose one controlled failure without copying the solution.
10. Complete the evidence checklist and make a PASS, REPAIR or STOP decision against the acceptance rule.

Lab-specific workflow checks

Sequence	Workflow stage	What to inspect
1	**Run baseline**	Inputs, changed state, permissions, observable status and retained verification evidence
2	**Score outputs**	Inputs, changed state, permissions, observable status and retained verification evidence
3	**Classify failure**	Inputs, changed state, permissions, observable status and retained verification evidence
4	**Change one layer**	Inputs, changed state, permissions, observable status and retained verification evidence

5	**Re-run set**	Inputs, changed state, permissions, observable status and retained verification evidence
6	**Decide**	Inputs, changed state, permissions, observable status and retained verification evidence

Prompt contract

> ***Use the baseline outputs and rubric to identify the dominant failure. Propose one scoped change and calculate the expected impact without changing the test set.***

Acceptance check

PASS ONLY WHEN

The change targets one causal layer, the frozen set is preserved, and regression risk is documented.

Troubleshooting

Observed issue	Likely cause	Remedial action
The agent invents a customer or market claim	Source IDs were omitted from the prompt or output contract	Repair the prompt and require row-level citations plus an uncertainty label.
The workflow recommends a tool without comparison	Criteria or common test case were not frozen	Run every option on the same mock input and apply the stated weights.
Output is plausible but unsupported	Acceptance omitted observable evidence	Require the claim ledger, scored rubric and named decision owner before acceptance.
Verifier reports a missing artifact	The required JSON/YAML field or output file is absent	Use brief.yaml as the contract, add only the missing field and re-run the verifier.

5.9 Activity 09: Select a Deployment Posture

OBJECTIVE

Compare channel and architecture options for an agentic product using task, integration and risk evidence.

Folder contents

- activities/activity-09-deployment-decision/README.md — detailed procedure
- README.pdf — detailed activity procedure and troubleshooting
- brief.yaml and prompt.md — machine-readable scenario and bounded task contract
- data/mock-data.csv and data/mock-context.json — synthetic inputs with source IDs
- scripts/verify_activity.py, solution/expected-output.json and evidence/checklist.md

Detailed procedure

1. Open the individual activity folder and read README.pdf, brief.yaml, prompt.md and evidence/checklist.md before starting.
2. Inspect the supplied mock CSV/JSON records and note their source IDs, observation date, limitations and missing fields.
3. Restate the product decision this activity must unlock and the observable acceptance rule.
4. Use prompt.md with an approved agentic AI builder; keep the role, data, tool and authority boundaries explicit.
5. Capture the plan, intermediate claims, handoff state and final artifact without introducing live or confidential data.
6. Trace every material claim to the supplied source rows and label inference, uncertainty and contradiction.
7. Review the artifact at the human gate; reject unsupported claims, unowned actions and uncontrolled automation.
8. Run `python3 scripts/verify_activity.py` and retain its JSON result with the activity evidence.
9. Compare the result against the worked solution and diagnose one controlled failure without copying the solution.
10. Complete the evidence checklist and make a PASS, REPAIR or STOP decision against the acceptance rule.

Lab-specific workflow checks

Sequence	Workflow stage	What to inspect
1	**Map context**	Inputs, changed state, permissions, observable status and retained verification evidence
2	**Compare channels**	Inputs, changed state, permissions, observable status and retained verification evidence
3	**Select architecture**	Inputs, changed state, permissions, observable status and retained verification evidence
4	**Add fallback**	Inputs, changed state, permissions, observable status and retained

		verification evidence
5	**Cost posture**	Inputs, changed state, permissions, observable status and retained verification evidence
6	**Approve**	Inputs, changed state, permissions, observable status and retained verification evidence

Prompt contract

> ***Evaluate the supplied web, workspace and messaging options. Recommend a deployment posture and explain rejected alternatives.***

Acceptance check

PASS ONLY WHEN

The selected option meets task, integration, safety and recovery thresholds; caveats are explicit.

Troubleshooting

Observed issue	Likely cause	Remedial action
The agent invents a customer or market claim	Source IDs were omitted from the prompt or output contract	Repair the prompt and require row-level citations plus an uncertainty label.
The workflow recommends a tool without comparison	Criteria or common test case were not frozen	Run every option on the same mock input and apply the stated weights.
Output is plausible but unsupported	Acceptance omitted observable evidence	Require the claim ledger, scored rubric and named decision owner before acceptance.
Verifier reports a missing artifact	The required JSON/YAML field or output file is absent	Use brief.yaml as the contract, add only the missing field and re-run the verifier.

5.10 Activity 10: Conduct a Risk and Control Review

OBJECTIVE

Threat-model an agent workflow and assign layered controls, evidence and owners.

Folder contents

- activities/activity-10-risk-control-review/README.md — detailed procedure
- README.pdf — detailed activity procedure and troubleshooting
- brief.yaml and prompt.md — machine-readable scenario and bounded task contract
- data/mock-data.csv and data/mock-context.json — synthetic inputs with source IDs
- scripts/verify_activity.py, solution/expected-output.json and evidence/checklist.md

Detailed procedure

1. Open the individual activity folder and read README.pdf, brief.yaml, prompt.md and evidence/checklist.md before starting.
2. Inspect the supplied mock CSV/JSON records and note their source IDs, observation date, limitations and missing fields.
3. Restate the product decision this activity must unlock and the observable acceptance rule.
4. Use prompt.md with an approved agentic AI builder; keep the role, data, tool and authority boundaries explicit.
5. Capture the plan, intermediate claims, handoff state and final artifact without introducing live or confidential data.
6. Trace every material claim to the supplied source rows and label inference, uncertainty and contradiction.
7. Review the artifact at the human gate; reject unsupported claims, unowned actions and uncontrolled automation.
8. Run `python3 scripts/verify_activity.py` and retain its JSON result with the activity evidence.
9. Compare the result against the worked solution and diagnose one controlled failure without copying the solution.
10. Complete the evidence checklist and make a PASS, REPAIR or STOP decision against the acceptance rule.

Lab-specific workflow checks

Sequence	Workflow stage	What to inspect
1	**Map assets**	Inputs, changed state, permissions, observable status and retained verification evidence
2	**Identify threats**	Inputs, changed state, permissions, observable status and retained verification evidence
3	**Score exposure**	Inputs, changed state, permissions, observable status and retained verification evidence
4	**Select controls**	Inputs, changed state, permissions, observable status and retained verification evidence

5	**Assign owner**	Inputs, changed state, permissions, observable status and retained verification evidence
6	**Test control**	Inputs, changed state, permissions, observable status and retained verification evidence

Prompt contract

> ***Use the supplied data-flow and incident cards to build a risk register. Apply least privilege and meaningful approval at consequential actions.***

Acceptance check

PASS ONLY WHEN

High risks have preventive and detective controls, named owners and testable evidence.

Troubleshooting

Observed issue	Likely cause	Remedial action
The agent invents a customer or market claim	Source IDs were omitted from the prompt or output contract	Repair the prompt and require row-level citations plus an uncertainty label.
The workflow recommends a tool without comparison	Criteria or common test case were not frozen	Run every option on the same mock input and apply the stated weights.
Output is plausible but unsupported	Acceptance omitted observable evidence	Require the claim ledger, scored rubric and named decision owner before acceptance.
Verifier reports a missing artifact	The required JSON/YAML field or output file is absent	Use brief.yaml as the contract, add only the missing field and re-run the verifier.

5.11 Activity 11: Design KPI and Feedback Dashboard

OBJECTIVE

Create a metric tree and monitoring view that balances business value, quality, experience, cost and risk.

Folder contents

- activities/activity-11-kpi-feedback-dashboard/README.md — detailed procedure
- README.pdf — detailed activity procedure and troubleshooting
- brief.yaml and prompt.md — machine-readable scenario and bounded task contract
- data/mock-data.csv and data/mock-context.json — synthetic inputs with source IDs
- scripts/verify_activity.py, solution/expected-output.json and evidence/checklist.md

Detailed procedure

1. Open the individual activity folder and read README.pdf, brief.yaml, prompt.md and evidence/checklist.md before starting.
2. Inspect the supplied mock CSV/JSON records and note their source IDs, observation date, limitations and missing fields.
3. Restate the product decision this activity must unlock and the observable acceptance rule.
4. Use prompt.md with an approved agentic AI builder; keep the role, data, tool and authority boundaries explicit.
5. Capture the plan, intermediate claims, handoff state and final artifact without introducing live or confidential data.
6. Trace every material claim to the supplied source rows and label inference, uncertainty and contradiction.
7. Review the artifact at the human gate; reject unsupported claims, unowned actions and uncontrolled automation.
8. Run `python3 scripts/verify_activity.py` and retain its JSON result with the activity evidence.
9. Compare the result against the worked solution and diagnose one controlled failure without copying the solution.
10. Complete the evidence checklist and make a PASS, REPAIR or STOP decision against the acceptance rule.

Lab-specific workflow checks

Sequence	Workflow stage	What to inspect
1	**Set outcome**	Inputs, changed state, permissions, observable status and retained verification evidence
2	**Add guardrails**	Inputs, changed state, permissions, observable status and retained verification evidence
3	**Define events**	Inputs, changed state, permissions, observable status and retained verification evidence
4	**Set thresholds**	Inputs, changed state, permissions, observable status and retained

		verification evidence
5	**Assign action**	Inputs, changed state, permissions, observable status and retained verification evidence
6	**Review drift**	Inputs, changed state, permissions, observable status and retained verification evidence

Prompt contract

> ***Design KPIs from the supplied event dictionary and baseline. Include leading indicators, guardrails and an owner for each alert.***

Acceptance check

PASS ONLY WHEN

Metrics have formulas, data sources, thresholds, segments and action owners; no vanity-only measure drives release.

Troubleshooting

Observed issue	Likely cause	Remedial action
The agent invents a customer or market claim	Source IDs were omitted from the prompt or output contract	Repair the prompt and require row-level citations plus an uncertainty label.
The workflow recommends a tool without comparison	Criteria or common test case were not frozen	Run every option on the same mock input and apply the stated weights.
Output is plausible but unsupported	Acceptance omitted observable evidence	Require the claim ledger, scored rubric and named decision owner before acceptance.
Verifier reports a missing artifact	The required JSON/YAML field or output file is absent	Use brief.yaml as the contract, add only the missing field and re-run the verifier.

5.12 Activity 12: Deliver a Governed Product Recommendation

OBJECTIVE

Synthesize value, evaluation, readiness and risk evidence into an executive pilot, expand, hold or stop decision.

Folder contents

- activities/activity-12-executive-recommendation/README.md — detailed procedure
- README.pdf — detailed activity procedure and troubleshooting
- brief.yaml and prompt.md — machine-readable scenario and bounded task contract
- data/mock-data.csv and data/mock-context.json — synthetic inputs with source IDs
- scripts/verify_activity.py, solution/expected-output.json and evidence/checklist.md

Detailed procedure

1. Open the individual activity folder and read README.pdf, brief.yaml, prompt.md and evidence/checklist.md before starting.
2. Inspect the supplied mock CSV/JSON records and note their source IDs, observation date, limitations and missing fields.
3. Restate the product decision this activity must unlock and the observable acceptance rule.
4. Use prompt.md with an approved agentic AI builder; keep the role, data, tool and authority boundaries explicit.
5. Capture the plan, intermediate claims, handoff state and final artifact without introducing live or confidential data.
6. Trace every material claim to the supplied source rows and label inference, uncertainty and contradiction.
7. Review the artifact at the human gate; reject unsupported claims, unowned actions and uncontrolled automation.
8. Run `python3 scripts/verify_activity.py` and retain its JSON result with the activity evidence.
9. Compare the result against the worked solution and diagnose one controlled failure without copying the solution.
10. Complete the evidence checklist and make a PASS, REPAIR or STOP decision against the acceptance rule.

Lab-specific workflow checks

Sequence	Workflow stage	What to inspect
1	**Summarise value**	Inputs, changed state, permissions, observable status and retained verification evidence
2	**Compare evidence**	Inputs, changed state, permissions, observable status and retained verification evidence
3	**Price risk**	Inputs, changed state, permissions, observable status and retained verification evidence
4	**Test downside**	Inputs, changed state, permissions, observable status and retained

		verification evidence
5	**Set conditions**	Inputs, changed state, permissions, observable status and retained verification evidence
6	**Recommend**	Inputs, changed state, permissions, observable status and retained verification evidence

Prompt contract

> ***Prepare an executive decision memo using only the supplied evidence pack. State the recommendation, conditions, residual risks and named owners.***

Acceptance check

PASS ONLY WHEN

The recommendation is traceable to evidence, survives a downside scenario, and has measurable exit criteria.

Troubleshooting

Observed issue	Likely cause	Remedial action
The agent invents a customer or market claim	Source IDs were omitted from the prompt or output contract	Repair the prompt and require row-level citations plus an uncertainty label.
The workflow recommends a tool without comparison	Criteria or common test case were not frozen	Run every option on the same mock input and apply the stated weights.
Output is plausible but unsupported	Acceptance omitted observable evidence	Require the claim ledger, scored rubric and named decision owner before acceptance.
Verifier reports a missing artifact	The required JSON/YAML field or output file is absent	Use brief.yaml as the contract, add only the missing field and re-run the verifier.

6. Agentic Product Control and Evidence Contract

An agentic product task is complete only when the requested state exists and the required checks demonstrate it. Preserve the product context, source evidence, agent and tool boundaries, handoff records, evaluation results and the named human decision at every consequential boundary.

DEFAULT EXECUTION POSTURE

Start with the simplest workflow and narrowest workable authority, use approved synthetic data, keep consequential actions behind explicit approval, and reject completion claims unsupported by retrievable evidence.

7. Assessment Preparation

The Written Assessment contains six open-ended questions aligned to K1–K6. The Practical Performance contains three tasks covering A1–A3. Use the activity evidence model: observed value, approved threshold, decision and named owner.

8. References

- **Current course page:** https://www.tertiarycourses.com.sg/wsq-agentic-ai-for-product-development.html — Course identity, duration, learning outcomes, topics and assessment format
- **AI Product Management:** Local reference/AI Product Management.pdf — AI-PM playground, natural-language development, rapid prototyping, specification, iteration and production handoff
- **Building AI-Powered Products:** Local reference/Building AI-Powered Products.pdf — AI product-development lifecycle, product sense, goals and metrics, agent patterns, evaluation and production rollout
- **Building effective agents:** https://www.anthropic.com/engineering/building-effective-agents — Agent/workflow distinction, simple composable patterns and complexity trade-offs
- **People + AI Guidebook:** https://pair.withgoogle.com/guidebook-v2/ — User needs, success definition, mental models, feedback, control and graceful failure
- **NIST AI RMF:** https://www.nist.gov/itl/ai-risk-management-framework — Govern, map, measure and manage risk throughout the AI lifecycle

Course repository: https://github.com/tertiarycourses/TGS-2024045799-Agentic-AI-for-Product-Development

Learning platform: https://lms-tms.tertiaryinfotech.com/